

- [6] J. E. Falk, "Lagrange multipliers and nonlinear programming," *J. Math. Anal. Appl.*, vol. 19, July 1967.
- [7] O. L. Mangasarian and J. Ponsstein, "Minimax and duality in nonlinear programming," *J. Math. Anal. Appl.*, vol. 11, pp. 504-518, 1965.
- [8] J. Stoer, "Duality in nonlinear programming and the minimax theorem," *Numerische Mathematik*, vol. 5, pp. 371-379, 1963.
- [9] R. T. Rockafellar, "Duality and stability in extremum problems involving convex functions," *Pacific J. Math.*, vol. 21, pp. 167-187, 1967.
- [10] P. Wolfe, "A duality theorem for nonlinear programming," *Q. Appl. Math.*, vol. 19, pp. 239-244, 1961.
- [11] R. T. Rockafellar, "Nonlinear programming," presented at the American Mathematical Society Summer Seminar on the Mathematics of the Decision Sciences, Stanford University, Stanford, Calif., July-August 1967.
- [12] D. G. Luenberger, "Convex programming and duality in normal space," *Proc. IEEE Systems Science and Cybernetics Conf.* (Boston, Mass., October 11-13, 1967).
- [13] J. M. Danskin, "The theory of max-min with applications," *J. SIAM*, vol. 14, pp. 641-665, July 1966.
- [14] W. Fenchel, "Convex cones, sets, and functions," mimeographed notes, Princeton University, Princeton, N. J., September 1963.
- [15] R. Fletcher and M. J. D. Powell, "A rapidly convergent descent method for minimization," *Computer J.*, vol. 6, p. 163, July 1963.
- [16] L. S. Lasdon and A. D. Waren, "Mathematical programming for optimal design," *Electro-Technol.*, pp. 53-71, November 1967.
- [17] J. B. Rosen, "The gradient projection method for nonlinear programming, pt. I, linear constraints," *J. SIAM*, vol. 8, pp. 181-217, 1960.
- [18] R. Fletcher and C. M. Reeves, "Function minimization by conjugate gradients," *Computer J.*, vol. 7, July 1964.
- [19] D. Goldfarb, "A conjugate gradient method for nonlinear programming," Ph.D. dissertation, Dept. of Chem. Engrg., Princeton University, Princeton, N. J., 1966.
- [20] L. S. Lasdon, "A multi-level technique for optimization," Ph.D. dissertation, Systems Research Center, Case Institute of Technology, Cleveland, Ohio, Rept. SRC 50-C-64-19, 1964.
- [21] L. S. Lasdon and J. D. Schoeffler, "A multi-level technique for optimization," Preprints, Joint Automatic Control Conf., Troy, N. Y., June 22-25, 1965, pp. 85-92.
- [22] —, "Decentralized plant control," *ISA Trans.*, vol. 5, pp. 175-183, April 1966.
- [23] C. B. Brosilow and L. S. Lasdon, "A two level optimization technique for recycle processes," *1965 Proc. AIChE—Symp. on Application of Mathematical Models in Chemical Engineering Research, Design, and Production* (London, England).
- [24] L. S. Lasdon, "Duality and decomposition in mathematical programming," Systems Research Center, Case Institute of Technology, Cleveland, Ohio, Rept. SRC 119-C-67-52, 1967.
- [25] A. V. Fiacco and G. P. McCormick, *Sequential Unconstrained Minimization Techniques for Nonlinear Programming*. New York: Wiley, 1968.
- [26] R. Fox and L. Schmit, "Advances in the integrated approach to structural synthesis," *J. Spacecraft and Rockets*, vol. 3, p. 858, June 1966.
- [27] B. P. Dzielinski and R. E. Gomory, "Optimal programming of lot sizes, inventory, and labor allocations," *Management Sci.*, vol. 11, pp. 874-890, July 1965.
- [28] J. E. Falk, "A relaxed interior approach to nonlinear programming," Research Analysis Corp., McLean, Va. RAC-TP-279, 1967.

A Formal Basis for the Heuristic Determination of Minimum Cost Paths

PETER E. HART, MEMBER, IEEE, NILS J. NILSSON, MEMBER, IEEE, AND BERTRAM RAPHAEL

Abstract—Although the problem of determining the minimum cost path through a graph arises naturally in a number of interesting applications, there has been no underlying theory to guide the development of efficient search procedures. Moreover, there is no adequate conceptual framework within which the various ad hoc search strategies proposed to date can be compared. This paper describes how heuristic information from the problem domain can be incorporated into a formal mathematical theory of graph searching and demonstrates an optimality property of a class of search strategies.

I. INTRODUCTION

A. The Problem of Finding Paths Through Graphs

MANY PROBLEMS of engineering and scientific importance can be related to the general problem of finding a path through a graph. Examples of such problems include routing of telephone traffic, navigation through a maze, layout of printed circuit boards, and

mechanical theorem-proving and problem-solving. These problems have usually been approached in one of two ways, which we shall call the *mathematical approach* and the *heuristic approach*.

1) The mathematical approach typically deals with the properties of abstract graphs and with algorithms that prescribe an orderly examination of nodes of a graph to establish a minimum cost path. For example, Pollock and Wiebenson^[1] review several algorithms which are guaranteed to find such a path for any graph. Busacker and Saaty^[2] also discuss several algorithms, one of which uses the concept of dynamic programming.^[3] The mathematical approach is generally more concerned with the ultimate achievement of solutions than it is with the computational feasibility of the algorithms developed.

2) The heuristic approach typically uses special knowledge about the domain of the problem being represented by a graph to improve the computational efficiency of solutions to particular graph-searching problems. For example, Gelernter's^[4] program used Euclidean diagrams to direct the search for geometric proofs. Samuel^[5] and others have used ad hoc characteristics of particular games to reduce

Manuscript received November 24, 1967.

The authors are with the Artificial Intelligence Group of the Applied Physics Laboratory, Stanford Research Institute, Menlo Park, Calif.

the “look-ahead” effort in searching game trees. Procedures developed via the heuristic approach generally have not been able to guarantee that minimum cost solution paths will always be found.

This paper draws together the above two approaches by describing how information from a problem domain can be incorporated in a formal mathematical approach to a graph analysis problem. It also presents a general algorithm which prescribes how to use such information to find a minimum cost path through a graph. Finally, it proves, under mild assumptions, that this algorithm is optimal in the sense that it examines the smallest number of nodes necessary to guarantee a minimum cost solution.

The following is a typical illustration of the sort of problem to which our results are applicable. Imagine a set of cities with roads connecting certain pairs of them. Suppose we desire a technique for discovering a sequence of cities on the shortest route from a specified start to a specified goal city. Our algorithm prescribes how to use special knowledge—e.g., the knowledge that the shortest road route between any pair of cities cannot be less than the airline distance between them—in order to reduce the total number of cities that need to be considered.

First, we must make some preliminary statements and definitions about graphs and search algorithms.

B. Some Definitions About Graphs

A *graph* G is defined to be a set $\{n_i\}$ of elements called *nodes* and a set $\{e_{ij}\}$ of directed line segments called *arcs*. If e_{pq} is an element of the set $\{e_{ij}\}$, then we say that there is an *arc* from node n_p to node n_q and that n_q is a *successor* of n_p . We shall be concerned here with graphs whose arcs have *costs* associated with them. We shall represent the cost of arc e_{ij} by c_{ij} . (An arc from n_i to n_j does not imply the existence of an arc from n_j to n_i . If both arcs exist, in general $c_{ij} \neq c_{ji}$.) We shall consider only those graphs G for which there exists $\delta > 0$ such that the cost of every arc of G is greater than or equal to δ . Such graphs shall be called δ *graphs*.

In many problems of interest the graph is not specified explicitly as a set of nodes and arcs, but rather is specified implicitly by means of a set of source nodes $S \subseteq \{n_i\}$ and a successor operator Γ , defined on $\{n_i\}$, whose value for each n_i is a set of pairs $\{(n_j, c_{ij})\}$. In other words, applying Γ to node n_i yields all the successors n_j of n_i and the costs c_{ij} associated with the arcs from n_i to the various n_j . Application of Γ to the source nodes, to their successors, and so forth as long as new nodes can be generated results in an explicit specification of the graph thus defined. We shall assume throughout this paper that a graph G is always given in implicit form.

The subgraph G_n from any node n in $\{n_i\}$ is the graph defined implicitly by the single source node n and some Γ defined on $\{n_i\}$. We shall say that each node in G_n is *accessible* from n .

A *path* from n_1 to n_k is an ordered set of nodes $(n_1, n_2, \dots, n_k)$ with each n_{i+1} a successor of n_i . There exists a path from n_i to n_j if and only if n_j is accessible from n_i . Every

path has a cost which is obtained by adding the individual costs of each arc, $c_{i,i+1}$, in the path. An *optimal path* from n_i to n_j is a path having the smallest cost over the set of all paths from n_i to n_j . We shall represent this cost by $h(n_i, n_j)$.

This paper will be concerned with the subgraph G_s from some single specified *start node* s . We define a nonempty set T of nodes in G_s as the *goal nodes*.¹ For any node n in G_s , an element $t \in T$ is a *preferred* goal node of n if and only if the cost of an optimal path from n to t does not exceed the cost of any other path from n to any member of T . For simplicity, we shall represent the unique cost of an optimal path from n to a preferred goal node of n by the symbol $h(n)$; i.e., $h(n) = \min_{t \in T} h(n, t)$.

C. Algorithms for Finding Minimum Cost Paths

We are interested in algorithms that search G_s to find an optimal path from s to a preferred goal node of s . What we mean by searching a graph and finding an optimal path is made clear by describing in general how such algorithms proceed. Starting with the node s , they generate some part of the subgraph G_s by repetitive application of the successor operator Γ . During the course of the algorithm, if Γ is applied to a node, we say that the algorithm has *expanded* that node.

We can keep track of the minimum cost path from s to each node encountered as follows. Each time a node is expanded, we store with each successor node n both the cost of getting to n by the lowest cost path found thus far, and a pointer to the predecessor of n along that path. Eventually the algorithm terminates at some goal node t , and no more nodes are expanded. We can then reconstruct a minimum cost path from s to t known at the time of termination simply by chaining back from t to s through the pointers.

We call an algorithm *admissible* if it is guaranteed to find an optimal path from s to a preferred goal node of s for any δ graph. Various admissible algorithms may differ both in the order in which they expand the nodes of G_s and in the number of nodes expanded. In the next section, we shall propose a way of ordering node expansion and show that the resulting algorithm is admissible. Then, in a following section, we shall show, under a mild assumption, that this algorithm uses information from the problem represented by the graph in an optimal way. That is, it expands the smallest number of nodes necessary to guarantee finding an optimal path.

II. AN ADMISSIBLE SEARCHING ALGORITHM

A. Description of the Algorithm

In order to expand the fewest possible nodes in searching for an optimal path, a search algorithm must constantly make as informed a decision as possible about which node to expand next. If it expands nodes which obviously cannot be on an optimal path, it is wasting effort. On the other hand, if it continues to ignore nodes that might be on an

¹ We exclude the trivial case of $s \in T$.